

Group Anagrams : Given an array of strings, group together strings that are anagrams (words having identical character frequency distributions), Example Input: ["eat","tea","tan","ate","nat","bat"] → Output: [["eat","tea","ate"],["tan","nat"],["bat"]]; optimal approach uses a Hash Map where key is sorted string or a 26-length frequency tuple and value is list of strings sharing that key, Time Complexity $O(n * k \log k)$ if sorting each string of length k , Space Complexity $O(n)$.

Prerequisites

Before attempting this problem, you should be comfortable with:

- **Hash Maps/Dictionaries** - Grouping items by a common key and storing lists as values
- **String Manipulation** - Iterating through characters and building new strings
- **Sorting** - Understanding how sorting can normalize strings for comparison
- **Character Frequency Counting** - Using arrays or maps to count occurrences of each character
- **Hash Keys** - Creating immutable, hashable keys from mutable data (e.g., tuples from lists)

2. Hash Table

Intuition

Instead of sorting each string, we can represent every string by the frequency of its characters.

Since the problem uses lowercase English letters, a fixed-size array of length 26 can capture how many times each character appears.

Two strings are anagrams if and only if their frequency arrays are identical.

By using this frequency array (converted to a tuple so it can be a dictionary key), we can group all strings that share the same character counts.

Algorithm

1. Create a hash map where each key is a 26-length tuple representing character frequencies, and each value is a list of strings belonging to that anagram group.
2. For each string in the input:
 - Initialize a count array of size 26 with all zeros.
 - For each character c in the string, increment the count at the corresponding index.
 - Convert the count array to a tuple and use it as the key.
 - Append the string to the list associated with this key.
3. After processing all strings, return all the lists stored in the hash map.

class Solution:

```
def groupAnagrams(self, strs: List[str]) -> List[List[str]]:
    res = defaultdict(list)
    for s in strs:
        count = [0] * 26
        for c in s:
            count[ord(c) - ord('a')] += 1
        res[tuple(count)].append(s)
    return list(res.values())
```

Time & Space Complexity

- Time complexity: $O(m*n)O(m*n)$
- Space complexity:
 - $O(m)O(m)$ extra space.
 - $O(m*n)O(m*n)$ space for the output list.

Where mm is the number of strings and nn is the length of the longest string.

```
func groupAnagrams(strs []string) [][]string {
    res := make(map[[26]int][]string)

    for _, s := range strs {
        var count [26]int
        for _, c := range s {
            count[c-'a']++
        }
        res[count] = append(res[count], s)
    }

    var result [][]string
    for _, group := range res {
        result = append(result, group)
    }
    return result
}
```

Common Pitfalls

Using a Mutable Key Type for the Hash Map

When using character frequency arrays as keys, you must convert them to an immutable type (like a tuple in Python or a string in other languages). Lists and arrays are mutable and cannot be used as dictionary keys directly.

Wrong: using list as key

```
count = [0] * 26
```

```
res[count].append(s) # TypeError: unhashable type 'list'
```

Correct: convert to tuple

```
res[tuple(count)].append(s)
```

Assuming Input Contains Only Lowercase Letters

The frequency array approach with size 26 only works for lowercase English letters. If the input could contain uppercase letters or other characters, the solution would fail or produce incorrect groupings.

Wrong: will crash on uppercase

```
count[ord(c) - ord('a')] += 1 # Negative index for uppercase
```

Should validate or handle: $\text{ord}('A') - \text{ord}('a') = -32$

Creating a New Key Format That Has Collisions

When converting frequency counts to strings, using a naive format like concatenation without separators can cause collisions. For example, counts $[1,11]$ and $[11,1]$ could produce the same string "111".

Wrong: potential collisions

```
key = ''.join(str(c) for c in count) # "111" for both [1,11] and [11,1]
```

Correct: use separator

```
key = ','.join(str(c) for c in count) # "1,11" vs "11,1"
```

Hashing M Longest Consecutive Sequence

Longest Consecutive Sequence : Given unsorted array of integers, find length of longest sequence of consecutive numbers (numbers increasing by 1), Example Input: $[100,4,200,1,3,2]$ → Output: 4 because sequence $[1,2,3,4]$; optimal approach inserts elements into Hash Set and only starts counting when current number has no predecessor ($\text{num}-1$ not in set), ensuring each element visited once, Time Complexity $O(n)$, Space Complexity $O(n)$.

Prerequisites

Before attempting this problem, you should be comfortable with:

- **Hash Sets** - $O(1)$ average lookup time and membership testing for efficient sequence detection
- **Hash Maps** - Storing and updating sequence boundary lengths for the union-find-like approach
- **Sorting** - Understanding how sorting groups consecutive numbers together

3. Hash Set

Intuition

To avoid repeatedly recounting the same sequences, we only want to start counting when we find the **beginning** of a consecutive sequence.

A number is the start of a sequence if $\text{num} - 1$ is **not** in the set.

This guarantees that each consecutive sequence is counted exactly once.

Once we identify such a starting number, we simply keep checking if $\text{num} + 1$, $\text{num} + 2$, ... exist in the set and extend the streak as far as possible.

This makes the solution efficient and clean because each number contributes to the sequence only one time.

Algorithm

1. Convert the list into a set `numSet` for $O(1)$ lookups.
2. Initialize `longest` to track the length of the longest consecutive sequence.
3. For each number `num` in `numSet`:
 - Check if $\text{num} - 1$ is **not** in the set:
 - If true, `num` is the start of a sequence.
 - Initialize `length = 1`.
 - While $\text{num} + \text{length}$ exists in the set, increase `length`.
 - Update `longest` with the maximum length found.
4. Return `longest` after scanning all numbers.

class Solution:

```
def longestConsecutive(self, nums: List[int]) -> int:
    numSet = set(nums)
    longest = 0
```

```
    for num in numSet:
```

```
        if (num - 1) not in numSet:
```

```
            length = 1
```

```
            while (num + length) in numSet:
```

```
                length += 1
```

```

        longest = max(length, longest)
    }
    return longest

```

Time & Space Complexity

- Time complexity: $O(n)O(n)$
- Space complexity: $O(n)O(n)$

```

func longestConsecutive(nums []int) int {
    numSet := make(map[int]struct{})
    for _, num := range nums {
        numSet[num] = struct{}{}
    }

    longest := 0
    for num := range numSet {
        if _, found := numSet[num-1]; !found {
            length := 1
            for {
                if _, exists := numSet[num+length]; exists {
                    length++
                } else {
                    break
                }
            }
            if length > longest {
                longest = length
            }
        }
    }
    return longest
}

```

Common Pitfalls

Starting a Sequence from Every Number

A common inefficiency is to start counting a sequence from every number in the array, which leads to $O(n^2)$ time complexity. The key optimization is to only start counting from numbers that are the beginning of a sequence (i.e., $num - 1$ is not in the set).

Not Handling Duplicates Properly

The input array may contain duplicate values. Using a set automatically handles this, but if you iterate over the original array instead of the set, you may process the same sequence multiple times, wasting computation.

Forgetting to Handle Empty Input

When the input array is empty, the longest consecutive sequence has length 0. Some implementations that assume at least one element exists may fail or return incorrect results for this edge case.

Hashing E Two Sum

Two Sum : Given an array and a target integer, return indices of two distinct elements whose sum equals target; Example Input: `nums = [2,7,11,15]`, `target = 9` → Output: `[0,1]`; optimal approach uses Hash Map (key-value structure with $O(1)$ average lookup) storing value → index while checking if complement (`target - current value`) already exists; Time Complexity $O(n)$; Space Complexity $O(n)$.

Prerequisites

Before attempting this problem, you should be comfortable with:

- **Hash Maps** - Using dictionaries to store values and their indices for $O(1)$ lookup of complements
- **Two Pointers technique** - Understanding how sorting enables the two-pointer approach for finding pairs
- **Array traversal** - Iterating through arrays while tracking indices and values

4. Hash Map (One Pass)

Intuition

We can solve the problem in a single pass by iterating through the array and checking if the complement of the current element exists in the hash map. If it does, we return the indices of the current element and its complement. If not, we store the current element in the hash map. This guarantees that we will never use the same element twice, but we still check every element in the array.

Algorithm

1. Create a hash map to store the value and index of each element in the array.
2. Iterate through the array using index `i` and compute the complement of the current element, which is `target - nums[i]`.
3. Check if the complement exists in the hash map.
4. If it does, return the indices of the current element and its complement.
5. If no such pair is found, return an empty array.

[class Solution:](#)

```
def twoSum(self, nums: List[int], target: int) -> List[int]:
    prevMap = {} # val -> index

    for i, n in enumerate(nums):
        diff = target - n
        if diff in prevMap:
            return [prevMap[diff], i]
        prevMap[n] = i
```

Time & Space Complexity

- Time complexity: $O(n)O(n)$
- Space complexity: $O(n)O(n)$

```
func twoSum(nums []int, target int) []int {
    prevMap := make(map[int]int)

    for i, n := range nums {
        diff := target - n
        if j, found := prevMap[diff]; found {
            return []int{j, i}
        }
        prevMap[n] = i
    }
    return []int{}
}
```

Common Pitfalls

Using the Same Element Twice

You cannot use the same element twice to form a pair. When using a hash map, ensure you check that the found index differs from the current index, or use one-pass where you only look at previously seen elements.

Wrong: might return [i, i] when $\text{nums}[i] * 2 == \text{target}$

if diff in indices:

```
    return [i, indices[diff]]
```

Correct: ensure different indices

if diff in indices and $\text{indices}[\text{diff}] \neq i$:

```
    return [i, indices[diff]]
```

Returning Values Instead of Indices

The problem asks for indices, not the values themselves. A common mistake is returning the two numbers that sum to the target rather than their positions in the array.

Handling Duplicate Values

When building a hash map with values as keys, duplicate values overwrite earlier indices. In the two-pass approach, this still works because you check $\text{indices}[\text{diff}] \neq i$. In the one-pass approach, you check for the complement before inserting the current element.

Wrong Complement Calculation

The complement should be $\text{target} - \text{nums}[i]$, not $\text{nums}[i] - \text{target}$. Getting this backwards will search for the wrong value.

Hashing E

Valid Anagram

Valid Anagram : Given two strings (a sequence of characters) s and t , determine if t is an anagram (a word formed by rearranging letters of another word using exactly same character counts) of s ; Example Input: $s = \text{"anagram"}, t = \text{"nagaram"} \rightarrow$ Output: true; optimal approach counts character frequencies using a fixed-size array of length 26 for lowercase English letters (frequency array stores count of each character) and compares counts; Time Complexity $O(n)$; Space Complexity $O(1)$ because alphabet size is constant.

Prerequisites

Before attempting this problem, you should be comfortable with:

- **Hash Maps** - Using dictionaries or hash tables to count character frequencies
- **Sorting** - Sorting strings or arrays to compare elements in a consistent order
- **Arrays** - Using fixed-size arrays as an efficient alternative to hash maps for limited character sets

2. Hash Map

Intuition

If two strings are anagrams, they must use the same characters with the same frequencies.

Instead of sorting, we can count how many times each character appears in both strings.

By using two hash maps (or dictionaries), we track the frequency of every character in each string.

If both frequency maps match exactly, then the strings contain the same characters with same frequencies, meaning they are anagrams.

Algorithm

1. If the two strings have different lengths, return false immediately.
2. Create two hash maps to store character frequencies for each string.

3. Iterate through both strings at the same time:
 - Increase the character count for $s[i]$ in the first map.
 - Increase the character count for $t[i]$ in the second map.
4. After building both maps, compare them:
 - If the maps are equal, return true.
 - Otherwise, return false.

class Solution:

```
def isAnagram(self, s: str, t: str) -> bool:
    if len(s) != len(t):
        return False

    countS, countT = {}, {}

    for i in range(len(s)):
        countS[s[i]] = 1 + countS.get(s[i], 0)
        countT[t[i]] = 1 + countT.get(t[i], 0)
    return countS == countT
```

Time & Space Complexity

- Time complexity: $O(n+m)O(n+m)$
- Space complexity: $O(1)O(1)$ since we have at most 2626 different characters.

Where nn is the length of string ss and mm is the length of string tt .

```
func isAnagram(s string, t string) bool {
    if len(s) != len(t) {
        return false
    }

    countS, countT := make(map[rune]int), make(map[rune]int)
    for i, ch := range s {
        countS[ch]++
        countT[rune(t[i])]++
    }

    for k, v := range countS {
        if countT[k] != v {
            return false
        }
    }
    return true
}
```

Common Pitfalls

Forgetting to Check Length First

If two strings have different lengths, they cannot be anagrams. Skipping this early check means wasting time processing strings that could never match. Always compare lengths first and return false immediately if they differ.

Case Sensitivity Issues

When the problem specifies lowercase letters only (as in this problem), case sensitivity is not an issue. However, if the problem allows mixed case, forgetting to normalize to the same case (e.g., converting both strings to lowercase) will cause incorrect results where "Listen" and "Silent" would wrongly be considered non-anagrams.

Hashing M Top K Frequent Elements

Top K Frequent Elements : Given integer array $nums$ and integer k , return k most frequent elements (frequency means number of occurrences), Example Input: $nums = [1,1,1,2,2,3]$, $k = 2 \rightarrow$ Output: $[1,2]$; optimal approach first builds frequency map then uses Bucket Sort (array indexed by frequency where index represents occurrence count) to achieve linear time without full sorting, Time Complexity $O(n)$, Space Complexity $O(n)$.

Prerequisites

Before attempting this problem, you should be comfortable with:

- **Hash Maps** - Using dictionaries to count element frequencies efficiently
- **Sorting** - Sorting elements by custom criteria (frequency in this case)
- **Heap/Priority Queue** - Using min-heaps to maintain the top k elements efficiently
- **Bucket Sort** - Understanding how to use array indices as buckets for counting-based sorting

3. Bucket Sort Intuition

Each number in the array appears a certain number of times, and the maximum possible frequency is the length of the array.

We can use this idea by creating a list where the index represents a frequency, and at each index we store all numbers that appear exactly that many times.

For example:

- All numbers that appear 1 time go into group `freq[1]`.
- All numbers that appear 2 times go into group `freq[2]`.
- And so on.

After we build these groups, we look from the highest possible frequency down to the lowest and collect numbers from these groups until we have `k` of them.

This way, we directly jump to the most frequent numbers without sorting all the elements by frequency.

Algorithm

1. Build a frequency map that counts how many times each number appears.
2. Create a list of groups `freq`, where `freq[i]` will store all numbers that appear exactly `i` times.
3. For each number and its frequency in the map, add the number to `freq[frequency]`.
4. Initialize an empty result list.
5. Loop from the largest possible frequency down to 1:
 - For each number in `freq[i]`, add it to the result list.
 - Once the result contains `k` numbers, return it.

class Solution:

```
def topKFrequent(self, nums: List[int], k: int) -> List[int]:
    count = {}
    freq = [[] for i in range(len(nums) + 1)]

    for num in nums:
        count[num] = 1 + count.get(num, 0)
    for num, cnt in count.items():
        freq[cnt].append(num)

    res = []
    for i in range(len(freq) - 1, 0, -1):
        for num in freq[i]:
            res.append(num)
            if len(res) == k:
                return res
```

Time & Space Complexity

- Time complexity: $O(n)O(n)$
- Space complexity: $O(n)O(n)$

```
func topKFrequent(nums []int, k int) []int {
    count := make(map[int]int)
    freq := make([][]int, len(nums)+1)

    for _, num := range nums {
        count[num]++
    }
    for num, cnt := range count {
        freq[cnt] = append(freq[cnt], num)
    }

    res := []int{}
    for i := len(freq) - 1; i > 0; i-- {
        for _, num := range freq[i] {
            res = append(res, num)
            if len(res) == k {
                return res
            }
        }
    }
    return res
}
```

Common Pitfalls

Using a Max-Heap Instead of Min-Heap

When keeping track of the top `k` elements, a min-heap of size `k` is needed so you can efficiently remove the smallest frequency when the heap exceeds size `k`. Using a max-heap requires storing all elements and then extracting `k` times, which is less efficient. The min-heap approach maintains only the `k` largest frequencies at any time.

Forgetting to Handle Ties in Frequency

When multiple numbers have the same frequency, the order in which they appear in the result may vary. Most problem statements accept any valid ordering, but some solutions incorrectly assume a specific order or break when frequencies are equal. Ensure your comparison function handles equal frequencies gracefully.

Off-By-One in Bucket Sort Index

In bucket sort, frequencies range from 1 to n (the array length), so you need $n + 1$ buckets indexed 0 to n . A common mistake is creating only n buckets, causing an index out of bounds error when an element appears n times. Always allocate $\text{len}(\text{nums}) + 1$ buckets to accommodate all possible frequencies.

Hashing M Encode and Decode Strings (Leetcode Premium)

Encode and Decode Strings : Design algorithm to encode a list of strings into one string and decode back correctly (encoding must preserve boundaries and allow unambiguous recovery), Example Input: ["lint","code","love","you"] → Encoded string like "4#lint4#code4#love3#you" → Decoded Output: original list; optimal approach uses length-prefix encoding (store size before string followed by delimiter) ensuring deterministic parsing, Time Complexity $O(n)$, Space Complexity $O(1)$ excluding output.

Prerequisites

Before attempting this problem, you should be comfortable with:

- **String Manipulation** - Building strings by concatenation and extracting substrings based on indices
- **Delimiter Design** - Understanding how to choose separators that won't conflict with input content
- **Length-Prefix Encoding** - Using string lengths to unambiguously mark boundaries between encoded segments

2. Encoding & Decoding (Optimal)

Intuition

Instead of storing all string lengths first and then appending the strings, we can directly attach each string to its length.

For every string, we write `length#string`.

The `#` character acts as a clear boundary between the length and the actual content, and using the length ensures we know exactly how many characters to read—no matter what characters appear in the string itself.

During decoding, we simply read characters until we reach `#` to find the length, then extract exactly that many characters as the string.

This approach is both simpler and more efficient because it avoids building separate sections for lengths and content.

Algorithm

Encoding

1. Initialize an empty result string.
2. For each string in the list:
 - Compute its length.
 - Append `"length#string"` to the result.
3. Return the final encoded string.

Decoding

1. Initialize an empty list for the decoded strings and a pointer $i = 0$.
2. While i is within the bounds of the encoded string:
 - Move a pointer j forward until it finds `#` — this segment represents the length.
 - Convert the substring `s[i:j]` into an integer length.
 - Move i to the character right after `#`.
 - Extract the next length characters — this is the original string.
 - Append the extracted string to the result list.
 - Move i forward by length to continue decoding the next segment.
3. Return the list of decoded strings.

class Solution:

```
def encode(self, strs: List[str]) -> str:
    res = ""
    for s in strs:
        res += str(len(s)) + "#" + s
    return res
```

```
def decode(self, s: str) -> List[str]:
    res = []
    i = 0

    while i < len(s):
        j = i
        while s[j] != '#':
            j += 1
        length = int(s[i:j])
        res.append(s[j+1:j+1+length])
        i = j+1+length
```

```

i = j + 1
j = i + length
res.append(s[i:j])
i = j

```

```

return res

```

Expand

Time & Space Complexity

- Time complexity: $O(m)$ for each `encode()` and `decode()` function calls.
- Space complexity: $O(m+n)$ for each `encode()` and `decode()` function calls.

Where m is the sum of lengths of all the strings and n is the number of strings.

type Solution struct {}

```

func (s *Solution) Encode(strs []string) string {
    res := ""
    for _, str := range strs {
        res += strconv.Itoa(len(str)) + "#" + str
    }
    return res
}

```

```

func (s *Solution) Decode(encoded string) []string {
    res := []string{}
    i := 0
    for i < len(encoded) {
        j := i
        for encoded[j] != '#' {
            j++
        }
        length, _ := strconv.Atoi(encoded[i:j])
        i = j + 1
        res = append(res, encoded[i:i+length])
        i += length
    }
    return res
}

```

Common Pitfalls

Using a Delimiter That Can Appear in the Strings

Choosing a simple delimiter like a comma or space will break decoding if that character appears inside the original strings. The length-prefixing approach avoids this by using the length to know exactly how many characters to read, making the content irrelevant.

Not Handling Empty Strings or Empty Lists

Edge cases like an empty input list or strings that are themselves empty ("") require careful handling. Ensure your encoding distinguishes between an empty list and a list containing one empty string, and that decoding correctly reconstructs zero-length strings.

Parsing Length Incorrectly for Multi-Digit Numbers

When the length of a string is 10 or more, the length prefix becomes multi-digit. Ensure you read all digits before the # separator rather than just assuming a single digit. Using a loop to collect characters until reaching # handles this correctly.

Hashing M

Valid Sudoku

Valid Sudoku : Given partially filled 9×9 board, determine if it is valid (valid means each row, column, and 3×3 subgrid contains digits 1–9 without repetition), Example Input: standard partially filled board → Output: true or false; optimal approach uses Hash Sets to track seen digits per row, column, and box; Time Complexity $O(1)$ because board size fixed (81 cells), Space Complexity $O(1)$.

Prerequisites

Before attempting this problem, you should be comfortable with:

- **Hash Sets** - Used to track seen digits in rows, columns, and boxes for duplicate detection
- **2D Array Traversal** - Understanding how to iterate through a 9x9 grid and compute box indices
- **Bit Manipulation** - The optimized solution uses bitmasks to represent seen digits compactly

2. Hash Set (One Pass)

Intuition

Instead of checking rows, columns, and 3×3 boxes separately, we can validate the entire Sudoku board in **one single pass**.

For each cell, we check whether the digit has already appeared in:

1. the same **row**

2. the same **column**
3. the same **3×3 box**

We track these using three hash sets:

- rows[r] keeps digits seen in row r
- cols[c] keeps digits seen in column c
- squares[(r // 3, c // 3)] keeps digits in the 3×3 box

If a digit appears again in any of these places, the board is invalid.

Algorithm

1. Create three hash maps of sets:
 - rows to track digits in each row
 - cols to track digits in each column
 - squares to track digits in each 3×3 sub-box, keyed by (r // 3, c // 3)
2. Loop through every cell in the board:
 - Skip the cell if it contains ".".
 - Let val be the digit in the cell.
 - If val is already in:
 - rows[r] → duplicate in the row
 - cols[c] → duplicate in the column
 - squares[(r // 3, c // 3)] → duplicate in the 3×3 box
 Then return false.
3. Otherwise, add the digit to all three sets:
 - rows[r]
 - cols[c]
 - squares[(r // 3, c // 3)]
4. If the whole board is scanned without detecting duplicates, return true.

class Solution:

```
def isValidSudoku(self, board: List[List[str]]) -> bool:
    cols = defaultdict(set)
    rows = defaultdict(set)
    squares = defaultdict(set)

    for r in range(9):
        for c in range(9):
            if board[r][c] == ".":
                continue
            if (board[r][c] in rows[r]
                or board[r][c] in cols[c]
                or board[r][c] in squares[(r // 3, c // 3)]):
                return False

            cols[c].add(board[r][c])
            rows[r].add(board[r][c])
            squares[(r // 3, c // 3)].add(board[r][c])

    return True
```

Time & Space Complexity

- Time complexity: $O(n^2)O(n^2)$
- Space complexity: $O(n^2)O(n^2)$

```
func isValidSudoku(board [][]byte) bool {
    rows := make([]map[byte]bool, 9)
    cols := make([]map[byte]bool, 9)
    squares := make([]map[byte]bool, 9)

    for i := 0; i < 9; i++ {
        rows[i] = make(map[byte]bool)
        cols[i] = make(map[byte]bool)
        squares[i] = make(map[byte]bool)
    }

    for r := 0; r < 9; r++ {
        for c := 0; c < 9; c++ {
            if board[r][c] == '.' {
                continue
            }
            row := r / 3
            col := c / 3
            box := row * 3 + col
```

```

    }
    val := board[r][c]
    squareIdx := (r/3)*3 + c/3

    if rows[r][val] || cols[c][val] ||
       squares[squareIdx][val] {
        return false
    }

    rows[r][val] = true
    cols[c][val] = true
    squares[squareIdx][val] = true
  }
}

return true
}

```

Common Pitfalls

Wrong Box Index Calculation

The 3x3 box index formula $(r // 3) * 3 + (c // 3)$ is easy to get wrong. A common mistake is using $(r // 3, c // 3)$ as a tuple key but forgetting integer division, or computing $r // 3 + c // 3$ which doesn't uniquely identify boxes.

Wrong: doesn't uniquely identify boxes

box_idx = r // 3 + c // 3 # Box (0,1) and (1,0) both give 1

Correct: unique box index 0-8

box_idx = (r // 3) * 3 + c // 3

Not Skipping Empty Cells

Empty cells are represented by "." and should be skipped entirely. Forgetting to check for empty cells before processing will cause errors or incorrect duplicate detection.

Checking Validity vs Solvability

This problem only checks if the current board state is valid, not whether the puzzle is solvable. A board with no duplicates is valid even if it's impossible to complete.

Processing the Same Cell Multiple Times

When iterating through the board, make sure each cell is only processed once. Some implementations accidentally check the same digit multiple times when validating rows, columns, and boxes separately.

Hashing E Contains Duplicate

Contains Duplicate : Given an array (a contiguous memory structure storing elements of the same type indexed from 0) determine whether any value appears at least twice, meaning there exist indices $i \neq j$ such that $nums[i] = nums[j]$; Example Input: $nums = [1,2,3,1]$ → Output: true because value 1 repeats; optimal approach uses a Hash Set (a data structure that stores unique elements using a hash function which maps values to indices enabling average $O(1)$ insertion and lookup) by iterating once and checking if element already exists before inserting; Time Complexity $O(n)$ because each element is processed once; Space Complexity $O(n)$ in worst case if all elements are unique.

Prerequisites

Before attempting this problem, you should be comfortable with:

- **Hash Sets** - The optimal solution uses a hash set for $O(1)$ lookups to detect duplicates efficiently
- **Sorting** - An alternative approach sorts the array to bring duplicates adjacent, then scans linearly
- **Basic Array Traversal** - The brute force approach uses nested loops to compare all pairs of elements

3. Hash Set

Intuition

We can use a hash set to efficiently keep track of the values we have already encountered.

As we iterate through the array, we check whether the current value is already present in the set.

If it is, that means we've seen this value before, so a duplicate exists.

Using a hash set allows constant-time lookups, making this approach much more efficient than comparing every pair.

Algorithm

1. Initialize an empty hash set to store seen values.
2. Iterate through each number in the array.
3. For each number:
 - If it is already in the set, return true because a duplicate has been found.
 - Otherwise, add it to the set.
4. If the loop finishes without finding any duplicates, return false.

class Solution:

```
def hasDuplicate(self, nums: List[int]) -> bool:
```

```

seen = set()
for num in nums:
    if num in seen:
        return True
    seen.add(num)
return False

```

Time & Space Complexity

- Time complexity: $O(n)O(n)$
- Space complexity: $O(n)O(n)$

```

func hasDuplicate(nums []int) bool {
    seen := make(map[int]bool)
    for _, num := range nums {
        if seen[num] {
            return true
        }
        seen[num] = true
    }
    return false
}

```

Common Pitfalls

Using Wrong Comparison in Brute Force

When using nested loops, a common mistake is comparing an element with itself by starting the inner loop at i instead of $i + 1$.

Wrong: Compares element with itself

```

for i in range(len(nums)):
    for j in range(len(nums)): # Should start at i + 1
        if nums[i] == nums[j]:
            return True

```

Correct: Skip self-comparison

```

for i in range(len(nums)):
    for j in range(i + 1, len(nums)):
        if nums[i] == nums[j]:
            return True

```

Modifying Input Array Unexpectedly

The sorting approach modifies the original array, which may not be acceptable in some contexts. If the original order matters, make a copy first.

Caution: This modifies the input

```
nums.sort()
```

Safer: Sort a copy if original order matters

```
sorted_nums = sorted(nums)
```

Hashing M Insert Delete Get Random O(1)

Insert Delete Get Random O(1) : Design data structure supporting insert(val), remove(val), and getRandom() all in average O(1) time, Example Operations: insert(1) → true, remove(2) → false, insert(2) → true, getRandom() → 1 or 2 with equal probability; optimal approach combines Dynamic Array (resizable contiguous array allowing O(1) index access) and Hash Map mapping value to its index so deletion swaps with last element before popping, Time Complexity O(1) average per operation, Space Complexity O(n).

Prerequisites

Before attempting this problem, you should be comfortable with:

- **Hash Maps** - Understanding O(1) average-case insert, delete, and lookup operations
- **Dynamic Arrays** - Understanding how arrays support O(1) random access and O(1) amortized append/pop at the end
- **Swap-and-Pop Technique** - Knowing how to achieve O(1) deletion from an unordered collection by swapping with the last element

2. Hash Map + List

Intuition

To achieve O(1) for all operations including getRandom(), we combine a hash map with a dynamic array. The array stores the actual values and allows random access, while the hash map stores each value's index in the array for fast lookups.

The tricky part is deletion: removing from the middle of an array is O(n). We solve this by swapping the element to delete with the last element, then removing from the end in O(1) time.

This swap-and-pop technique is a common pattern for O(1) deletion from unordered collections.

Algorithm

1. Initialize a hash map numMap (value to index) and a list nums.
2. **insert(val)**: If val exists in the map, return false. Otherwise, add val to the end of the list and store its index in the map. Return true.

3. **remove(val)**: If val doesn't exist, return false. Get the index of val, swap it with the last element in the list, update the swapped element's index in the map, remove the last element from the list, and delete val from the map. Return true.
4. **getRandom()**: Return a random element from the list using a random index.

class RandomizedSet:

```
def __init__(self):
    self.numMap = {}
    self.nums = []

def insert(self, val: int) -> bool:
    if val in self.numMap:
        return False
    self.numMap[val] = len(self.nums)
    self.nums.append(val)
    return True

def remove(self, val: int) -> bool:
    if val not in self.numMap:
        return False
    idx = self.numMap[val]
    last = self.nums[-1]
    self.nums[idx] = last
    self.numMap[last] = idx
    self.nums.pop()
    del self.numMap[val]
    return True

def getRandom(self) -> int:
    return random.choice(self.nums)
```

Expand

Time & Space Complexity

- Time complexity: $O(1)$ for each function call.
- Space complexity: $O(n)$

type RandomizedSet struct {

```
    numMap map[int]int
    nums    []int
}
```

```
func Constructor() RandomizedSet {
    return RandomizedSet{
        numMap: make(map[int]int),
        nums:   []int{},
    }
}
```

```
func (this *RandomizedSet) Insert(val int) bool {
    if _, exists := this.numMap[val]; exists {
        return false
    }
    this.numMap[val] = len(this.nums)
    this.nums = append(this.nums, val)
    return true
}
```

```
func (this *RandomizedSet) Remove(val int) bool {
    if _, exists := this.numMap[val]; !exists {
        return false
    }
    idx := this.numMap[val]
    last := this.nums[len(this.nums)-1]
    this.nums[idx] = last
    this.numMap[last] = idx
    this.nums = this.nums[:len(this.nums)-1]
    delete(this.numMap, val)
    return true
}
```

```
func (this *RandomizedSet) GetRandom() int {
    return this.nums[rand.Intn(len(this.nums))]
}
```

Common Pitfalls

Not Updating the Swapped Element's Index

During removal, when swapping the element to delete with the last element, a critical step is updating the last element's index in the hash map before removing anything. Forgetting `numMap[last] = idx` causes the hash map to have a stale index for the swapped element, leading to incorrect behavior on subsequent operations involving that value.

Edge Case When Removing the Last Element

When the element to remove is already at the last position (i.e., `idx == len(nums) - 1`), the swap operation effectively swaps the element with itself. While this works correctly in most implementations, some code paths may have issues if they delete from the map before updating it. The order of operations matters: update the map first, then remove from both the list and map.

Using Hash Map Only Without Array

A common initial approach is to use only a hash map, which provides $O(1)$ insert and delete but $O(n)$ for `getRandom()` since hash maps do not support random index access. Converting keys to a list on each `getRandom()` call defeats the $O(1)$ requirement. The combination of hash map plus array is essential, where the array enables $O(1)$ random access and the hash map enables $O(1)$ lookup for the swap-and-pop deletion technique.

Hashing E Find the Difference of Two Arrays

Find the Difference of Two Arrays : Given two integer arrays `nums1` and `nums2`, return a list of two lists where the first contains elements present in `nums1` but not in `nums2` and the second contains elements present in `nums2` but not in `nums1` (set difference means elements existing in one set but not in another). Example Input: `nums1 = [1,2,3]`, `nums2 = [2,4,6]` → Output: `[[1,3],[4,6]]`; optimal approach converts both arrays into Hash Sets (data structure storing unique elements with $O(1)$ average lookup) and checks membership for each element, Time Complexity $O(n + m)$, Space Complexity $O(n + m)$.

Prerequisites

Before attempting this problem, you should be comfortable with:

- **Hash Sets** - The optimal solution uses sets for $O(1)$ lookup and set difference operations
- **Set Operations** - Understanding union, intersection, and difference of sets
- **Two Pointers** - Used in the sorting-based approach to efficiently compare sorted arrays

3. Hash Set

Intuition

Hash sets provide $O(1)$ average lookup time, making them ideal for membership testing. By converting both arrays to sets, we eliminate duplicates and can quickly check whether any element exists in the other array. This avoids the nested loops of the brute force approach.

Algorithm

1. Convert `nums1` and `nums2` into hash sets to remove duplicates and enable fast lookups.
2. Create an empty list for elements in `nums1` that aren't in `nums2`.
3. Iterate through the first set and add elements not found in the second set.
4. Repeat for elements in the second set that aren't in the first set.
5. Return both result lists.

class Solution:

```
def findDifference(self, nums1: List[int], nums2: List[int]) -> List[List[int]]:
    num1Set, num2Set = set(nums1), set(nums2)
    res1, res2 = [], []

    for num in num1Set:
        if num not in num2Set:
            res1.append(num)

    for num in num2Set:
        if num not in num1Set:
            res2.append(num)

    return [res1, res2]
```

Time & Space Complexity

- Time complexity: $O(n+m)O(n+m)$
- Space complexity: $O(n+m)O(n+m)$

Where nn is the size of the array `nums1` and mm is the size of the array `nums2`.

```
func findDifference(nums1 []int, nums2 []int) [][]int {
    num1Set := make(map[int]struct{})
    num2Set := make(map[int]struct{})
```

```

for _, num := range nums1 {
    num1Set[num] = struct{}{}
}
for _, num := range nums2 {
    num2Set[num] = struct{}{}
}

res1, res2 := []int{}, []int{}
for num := range num1Set {
    if _, exists := num2Set[num]; !exists {
        res1 = append(res1, num)
    }
}
for num := range num2Set {
    if _, exists := num1Set[num]; !exists {
        res2 = append(res2, num)
    }
}

return [][]int{res1, res2}
}

```

Common Pitfalls

Returning Duplicate Values in the Result

The problem asks for distinct integers. If you iterate through the original arrays instead of sets, you might add the same value multiple times to your result. Always use sets or track seen values to ensure each element appears only once in the output.

Confusing Which Array Each Result Belongs To

The output is a list of two lists: the first contains elements in nums1 but not in nums2, and the second contains elements in nums2 but not in nums1. Swapping these leads to incorrect results. Double-check which difference operation corresponds to which result list.

Hashing E Isomorphic Strings

Isomorphic Strings : Given two strings s and t, determine if characters in s can be replaced to get t using a bijection (one-to-one and onto mapping between characters); Example Input: s="egg", t="add" → Output: true; optimal approach uses two hash maps to maintain consistent forward and reverse mapping; Time Complexity O(n); Space Complexity O(1) fixed charset.

Prerequisites

Before attempting this problem, you should be comfortable with:

- **Hash Maps** - Storing and looking up key-value pairs in O(1) time
- **String Iteration** - Traversing characters in a string by index
- **Bijjective Mapping** - Understanding one-to-one correspondence between two sets

2. Hash Map (One Pass)

Intuition

We can verify both mapping directions simultaneously in a single pass. By maintaining two hash maps, one for s → t and one for t → s, we check at each position that neither mapping is violated. If a character in s was previously mapped to a different character in t, or if a character in t was previously mapped to a different character in s, the strings aren't isomorphic.

Algorithm

1. Create two hash maps: mapST for s → t and mapTS for t → s.
2. Iterate through both strings simultaneously:
 - If s[i] already maps to something other than t[i], return false.
 - If t[i] already maps to something other than s[i], return false.
 - Otherwise, record both mappings.
3. If the loop completes without conflicts, return true.

class Solution:

```

def isIsomorphic(self, s: str, t: str) -> bool:
    mapST, mapTS = {}, {}

    for i in range(len(s)):
        c1, c2 = s[i], t[i]
        if ((c1 in mapST and mapST[c1] != c2) or
            (c2 in mapTS and mapTS[c2] != c1)):
            return False
        mapST[c1] = c2
        mapTS[c2] = c1

```

```
return True
```

Time & Space Complexity

- Time complexity: $O(n)O(n)$
- Space complexity: $O(m)O(m)$

Where n is the length of the input string and m is the number of unique characters in the strings.

```
func isIsomorphic(s string, t string) bool {
    mapST := make(map[byte]byte)
    mapTS := make(map[byte]byte)

    for i := 0; i < len(s); i++ {
        c1, c2 := s[i], t[i]

        if val, ok := mapST[c1]; ok && val != c2 {
            return false
        }
        if val, ok := mapTS[c2]; ok && val != c1 {
            return false
        }

        mapST[c1] = c2
        mapTS[c2] = c1
    }

    return true
}
```

Common Pitfalls

Only Checking One Direction of Mapping

Checking only that each character in s maps to a unique character in t is insufficient. You must also verify the reverse: that each character in t maps to a unique character in s . For example, $s = "ab"$ and $t = "aa"$ would pass a one-way check but fails the bidirectional requirement since both 'a' and 'b' map to 'a'.

Assuming Same Character Sets

The two strings may contain completely different character sets. Do not assume that a character appearing in s must also appear in t , or vice versa. The mapping must be established dynamically as you iterate through both strings.

Hashing E Majority Element

Majority Element : Given array of size n , find element appearing more than $\lfloor n/2 \rfloor$ times; Example Input: $[3,2,3] \rightarrow$ Output: 3; optimal approach uses Boyer-Moore Voting Algorithm (algorithm maintaining candidate and count canceling out different elements) achieving $O(n)$ time and $O(1)$ space.

Prerequisites

Before attempting this problem, you should be comfortable with:

- **Hash Maps** - Counting element frequencies efficiently for $O(n)$ time solutions
- **Sorting** - Understanding that after sorting, the majority element occupies the middle position
- **Boyer-Moore Voting Algorithm** - The candidate elimination technique that finds majority elements in $O(1)$ space
- **Bit Manipulation** - Constructing numbers bit by bit by counting set bits across all elements (for the bit manipulation approach)

2. Hash Map

Intuition

We can avoid repeated counting by using a hash map to store the frequency of each element as we iterate through the array. We track the element with the maximum count seen so far. Once any element's count exceeds $n/2$, it must be the majority element.

Algorithm

1. Create a hash map to store element frequencies.
2. Initialize res and $maxCount$ to track the current best candidate.
3. For each element num :
 - Increment its count in the hash map.
 - If its count exceeds $maxCount$, update $res = num$ and $maxCount = count[num]$.
4. Return res .

class Solution:

```
def majorityElement(self, nums: List[int]) -> int:
    count = defaultdict(int)
    res = maxCount = 0
```

```

for num in nums:
    count[num] += 1
    if maxCount < count[num]:
        res = num
        maxCount = count[num]
return res

```

Time & Space Complexity

- Time complexity: $O(n)O(n)$
- Space complexity: $O(n)O(n)$

```

func majorityElement(nums []int) int {
    count := make(map[int]int)
    res, maxCount := 0, 0

```

```

    for _, num := range nums {
        count[num]++
        if count[num] > maxCount {
            res = num
            maxCount = count[num]
        }
    }
    return res
}

```

Common Pitfalls

Using $\geq$ Instead of $>$ for the Majority Check

The majority element must appear **more than** $n/2$ times, not $n/2$ or more. Using $\text{count} \geq n/2$ instead of $\text{count} > n/2$ can return incorrect results for arrays like $[1, 2, 2]$ where 2 appears exactly $n/2$ times but is not a strict majority.

Forgetting Integer Division Semantics

In languages like Python 3, $/$ performs floating-point division while $//$ performs integer division. Using $n / 2$ instead of $n // 2$ can lead to type errors or incorrect comparisons. Similarly, in JavaScript, $\text{Math.floor}(n / 2)$ is needed for proper integer division.

Hashing E Word Pattern

Word Pattern : Given pattern string and sentence, determine if pattern follows bijection with words; Example Input: $\text{pattern} = \text{"abba"}, \text{s} = \text{"dog cat cat dog"}$
 → Output: true; optimal approach splits string and uses two hash maps to ensure one-to-one mapping; Time Complexity $O(n)$; Space Complexity $O(n)$.

Prerequisites

Before attempting this problem, you should be comfortable with:

- **Hash Maps** - Using dictionaries for efficient key-value lookups and storing mappings
- **Bijection Concept** - Understanding one-to-one correspondence where each key maps to exactly one value and vice versa
- **String Manipulation** - Splitting strings by delimiters and iterating through characters

1. Two Hash Maps

Intuition

A valid pattern match requires a bijection (one-to-one correspondence) between pattern characters and words. Each character must map to exactly one word, and each word must map to exactly one character. Using two hash maps allows us to verify both directions of this mapping simultaneously as we iterate through the pattern and words.

Algorithm

1. Split the string s into an array of words.
2. If the pattern length does not equal the number of words, return false.
3. Create two maps: one mapping characters to words, another mapping words to characters.
4. For each character-word pair, check if existing mappings are consistent with the current pair.
5. If a conflict is found in either direction, return false. Otherwise, update both mappings.
6. Return true if all pairs are processed without conflicts.

class Solution:

```

def wordPattern(self, pattern: str, s: str) -> bool:
    words = s.split(" ")
    if len(pattern) != len(words):
        return False

```

```

    charToWord = {}
    wordToChar = {}

```

```

    for c, w in zip(pattern, words):
        if c in charToWord and charToWord[c] != w:

```



```

        return False
    if w in wordToChar and wordToChar[w] != c:
        return False
    charToWord[c] = w
    wordToChar[w] = c
    return True

```

Time & Space Complexity

- Time complexity: $O(n+m)O(n+m)$
- Space complexity: $O(m)O(m)$

Where nn is the length of the string *pattern* and mm is the length of the string *ss*.

```

func wordPattern(pattern string, s string) bool {
    words := strings.Split(s, " ")
    if len(pattern) != len(words) {
        return false
    }

    charToWord := make(map[byte]string)
    wordToChar := make(map[string]byte)

    for i := 0; i < len(pattern); i++ {
        c := pattern[i]
        w := words[i]

        if val, ok := charToWord[c]; ok && val != w {
            return false
        }
        if val, ok := wordToChar[w]; ok && val != c {
            return false
        }

        charToWord[c] = w
        wordToChar[w] = c
    }
    return true
}

```

Common Pitfalls

Only Checking One Direction of the Mapping

A common mistake is using only one hash map to check if each pattern character maps to the same word, but forgetting to verify the reverse. This fails cases like *pattern* = "abba" and *s* = "dog dog dog dog" where a -> dog and b -> dog would both be stored, incorrectly returning true.

Wrong: Only checks character -> word

```

if c in charToWord and charToWord[c] != w:
    return False

```

charToWord[c] = w # Missing reverse check!

Forgetting to Check Length Mismatch Early

Failing to compare the pattern length with the word count before iterating leads to index errors or incorrect results. For example, *pattern* = "ab" with *s* = "dog" has a mismatch that should immediately return false.

words = s.split()

Wrong: Missing this check before the loop

```

if len(pattern) != len(words):
    return False

```

Hashing E Find The Index of The First Occurrence in a String

Find The Index of The First Occurrence in a String : Given haystack and needle strings, return first index where needle appears in haystack or -1; Example Input: haystack="sadbutsad", needle="sad" → Output: 0; optimal approach uses KMP (Knuth-Morris-Pratt string matching algorithm using prefix table to avoid redundant comparisons) achieving $O(n+m)$ time and $O(m)$ space.

Prerequisites

Before attempting this problem, you should be comfortable with:

- **String Manipulation** - Basic operations like substring comparison and character indexing
- **KMP Algorithm (Optional)** - Understanding the Longest Proper Prefix Suffix (LPS) array for efficient pattern matching
- **Rolling Hash / Rabin-Karp (Optional)** - Using hash functions to compare substrings in $O(1)$ time

1. Brute Force

Intuition

The simplest way to find a substring is to try every possible starting position in the haystack. At each position, we compare characters one by one with the needle. If all characters match, we found our answer. If any character doesn't match, we move to the next starting position and try again.

Algorithm

1. Iterate through each valid starting position i in the haystack (from 0 to $n - m$, where n is the haystack length and m is the needle length).
2. For each starting position, compare characters of the haystack starting at i with characters of the needle.
3. If all characters match (we reach the end of the needle), return the starting position i .
4. If any character doesn't match, break out of the inner loop and try the next starting position.
5. If no match is found after checking all positions, return -1 .

class Solution:

```
def strStr(self, haystack: str, needle: str) -> int:
    n, m = len(haystack), len(needle)
    for i in range(n - m + 1):
        j = 0
        while j < m:
            if haystack[i + j] != needle[j]:
                break
            j += 1
        if j == m:
            return i
    return -1
```

Time & Space Complexity

- Time complexity: $O(n*m)$
- Space complexity: $O(1)$

Where n is the length of the string *haystack* and m is the length of the string *needle*.

```
func strStr(haystack string, needle string) int {
    n, m := len(haystack), len(needle)
    for i := 0; i <= n-m; i++ {
        j := 0
        for j < m {
            if haystack[i+j] != needle[j] {
                break
            }
            j++
        }
        if j == m {
            return i
        }
    }
    return -1
}
```

Common Pitfalls

Off-by-One Errors in Loop Bounds

When iterating through the haystack, the loop should run from 0 to $n - m$ inclusive. A common mistake is using $n - m - 1$ or $n - 1$ as the upper bound. The former misses the case where the needle appears at the very end, while the latter causes out-of-bounds access when comparing characters.

Not Handling Empty Needle

When the needle is an empty string, the expected return value is 0 (the empty string is found at the beginning of any string). Forgetting to check for this edge case can lead to incorrect behavior or infinite loops in some implementations.

Hashing E Ransom Note

Ransom Note : Given ransomNote and magazine strings, determine if ransomNote can be constructed using letters from magazine (each letter usable once); Example Input: ransomNote="aa", magazine="aab" → Output: true; optimal approach uses frequency counting array; Time Complexity $O(n)$; Space Complexity $O(1)$.

Prerequisites

Before attempting this problem, you should be comfortable with:

- **Hash Maps / Frequency Counting** - Used to efficiently count and compare character occurrences between two strings
- **String Iteration** - Understanding how to traverse characters in a string and perform lookups

2. Count Frequency

Intuition

Instead of searching and removing characters one by one, we can count the frequency of each character in both strings. The ransom note can be constructed if and only if the magazine contains at least as many of each character as the ransom note requires.

Algorithm

1. Count the frequency of each character in the ransom note (countR).
2. Count the frequency of each character in the magazine (countM).
3. For each character in countR:
 - If countM[c] < countR[c], return false.
4. Return true.

class Solution:

```
def canConstruct(self, ransomNote: str, magazine: str) -> bool:
    countR = Counter(ransomNote)
    countM = Counter(magazine)

    for c in countR:
        if countM[c] < countR[c]:
            return False

    return True
```

Time & Space Complexity

- Time complexity: $O(m+n)$
- Space complexity: $O(1)$ since we have at most 26 different characters.

Where m and n are the lengths of the strings `ransomNote` and `magazine`, respectively.

```
func canConstruct(ransomNote string, magazine string) bool {
    countR := make([]int, 26)
    countM := make([]int, 26)

    for _, c := range ransomNote {
        countR[c-'a']++
    }

    for _, c := range magazine {
        countM[c-'a']++
    }

    for i := 0; i < 26; i++ {
        if countM[i] < countR[i] {
            return false
        }
    }

    return true
}
```

Common Pitfalls

Counting the Wrong String First

A common mistake is counting characters from the ransom note first and then trying to match against the magazine. This leads to incorrect logic when checking availability. Always count the magazine characters first (the source), then verify against the ransom note requirements.

Not Handling Character Reuse Properly

When using the brute force approach, forgetting to remove a character from the magazine after using it leads to incorrect results. Each magazine character can only be used once, so after matching a character, it must be marked as consumed to prevent reuse.

Hashing M Determine if Two Strings Are Close

Determine if Two Strings Are Close: Two strings are close if they have same set of characters and frequency counts can be rearranged (meaning you can swap character frequencies), approach using frequency arrays and compare sorted frequency counts plus check character presence set equality, optimal time $O(n \log n)$ due to sorting frequency array (constant 26 actually so effectively $O(n)$), space $O(1)$.

Example:

Input: word1="abc", word2="bca"

Output: true

Hashing M Equal Row and Column Pairs

Equal Row and Column Pairs: Given $n \times n$ matrix, count pairs (row i , column j) where row equals column element-wise, approach store rows in hash map as string or tuple and compare with each column representation, optimal time $O(n^2)$ to build and compare, space $O(n^2)$ to store rows.

Example:

Input: [[3,2,1],[1,7,6],[2,7,7]]

Output: 1

Hashing E Unique Number of Occurrences

Unique Number of Occurrences: Given integer array, check if frequency of each value is unique, approach build frequency map then insert frequencies into set (data structure storing unique values) and compare sizes, optimal time $O(n)$, space $O(n)$.

Example:

Input: [1,2,2,1,1,3]

Output: true
